

Mini Project: เว็บแอปพลิเคชัน My Playlist

REST API ด้วย Node.js และ Express.js เชื่อมต่อกับหน้าเว็บ HTML, CSS และ JavaScript
จัดทำโดย

นายเมธี เอนกเมธิพิทกษา รหัสนักศึกษา 660910547

น.ส.นพรัตน์ ตันจินดา รหัสนักศึกษา 660911127

1. ที่มาและวัตถุประสงค์

โปรเจกต์นี้เป็นการฝึกพัฒนาเว็บแอปพลิเคชันแบบ full-stack ขนาดเล็ก โดยเลือกหัวข้อ (resource) เป็น เพลงและแนวเพลงที่ฉันชอบฟัง ฝั่งเซิร์ฟเวอร์ทำหน้าที่เก็บข้อมูลและให้บริการผ่าน REST API ส่วนฝั่งไคลเอนต์เป็นหน้าเว็บที่เรียก API ด้วย `fetch()` เพื่อแสดงและจัดการข้อมูลเพลง

วัตถุประสงค์

- เพื่อออกแบบและพัฒนา REST API ที่ทำงานได้ครบ CRUD (Create, Read, Update, Delete)
- เพื่อฝึกใช้ HTTP method และ status code ให้ถูกต้องตามความหมาย (200, 201, 204, 400, 404)
- เพื่อฝึกตรวจสอบข้อมูล (validation) ฝั่งเซิร์ฟเวอร์ก่อนบันทึก
- เพื่อเชื่อมต่อหน้าเว็บกับ API ด้วย `fetch()` และอัปเดตหน้าจอโดยไม่ต้องโหลดหน้าใหม่
- เพื่อฝึกจัดโครงสร้างโปรเจกต์ให้แยกฝั่งเซิร์ฟเวอร์และไคลเอนต์ชัดเจน และเขียนเอกสารประกอบ

2. ขอบเขตของงาน

ความสามารถของแอป

- แสดงรายชื่อเพลงทั้งหมด 33 เพลง พร้อมชื่อศิลปินและแนวเพลง โดยเรียงตามตัวอักษร A-Z เป็นค่าเริ่มต้น
- กรองเพลงตามแนวเพลง (7 แนว) ผ่านไอคอนโฟลเดอร์
- ค้นหาเพลงจากชื่อเพลงหรือชื่อศิลปิน และเรียงลำดับตามชื่อเพลงหรือชื่อศิลปิน
- เพิ่มเพลงใหม่ แก้ไขข้อมูลเพลง และลบเพลง
- กดที่เพลงเพื่อเปิดฟังใน Spotify ปุ่ม Play เปิดเพลงแรก และปุ่ม Shuffle สุ่มเพลง
- แจ้งผลการทำงาน (เพิ่ม / แก้ไข / ลบ) ด้วยการแจ้งเตือนรูปแบบ iOS ที่เลื่อนลงมาจากด้านบน

การตรวจสอบตามข้อกำหนดของงาน

ข้อกำหนด	สิ่งที่ทำในโปรเจกต์
GET /api/<resource> กรองด้วย query string อย่างน้อย 1 พารามิเตอร์	GET /api/songs รองรับ ?genre= ?artist= ?q= ?sort= (4 พารามิเตอร์)
GET /api/<resource>/:id ตอบ 404 ถ้าไม่พบ	GET /api/songs/:id ตอบ 404 พร้อมข้อความ error
POST ตรวจสอบข้อมูล ตอบ 400 / 201	ตรวจสอบ title, artist, genre, spotifyUrl ตอบ 400 พร้อมรายละเอียด หรือ 201 เมื่อสำเร็จ
PATCH หรือ PUT ตอบ 404 ถ้าไม่พบ	PATCH /api/songs/:id แก้บางฟิลด์ ตอบ 404 ถ้าไม่พบ
DELETE ตอบ 204 เมื่อสำเร็จ	DELETE /api/songs/:id ตอบ 204 No Content
แสดงรายการด้วย <code>fetch()</code> โดยไม่โหลดหน้าใหม่	<code>loadSongs()</code> เรียก GET แล้วสร้างรายการด้วย JavaScript
ฟอร์มเพิ่มรายการ เรียก POST แล้วอัปเดตทันที	ฟอร์มใน popup ส่ง POST แล้วเรียก <code>refresh()</code> ทันที
ปุ่มลบและแก้ไข เรียก DELETE และ PATCH/PUT	ปุ่ม 🗑 เรียก PATCH และปุ่ม 🗑 เรียก DELETE
ออกแบบสวยงาม ใช้งานง่าย	ธีมโมเดิร์นสีพาสเทลแบบหน้าจอสวย iOS จัดวาง 2 คอลัมน์
เรียก API ของตัวเองเท่านั้น	ทุก <code>fetch()</code> เรียก /api/... ของเซิร์ฟเวอร์ตัวเอง (Spotify เป็นเพียงลิงก์ให้ผู้ใช้กด)

ข้อกำหนด	สิ่งที่ทำในโปรเจกต์
แยกโฟลเดอร์ /server และ /public	แยกตามที่กำหนด
README.md อธิบายแอป วิธีติดตั้ง วิธีรัน และ endpoint	มีครบใน README.md

3. เครื่องมือและเทคโนโลยีที่ใช้

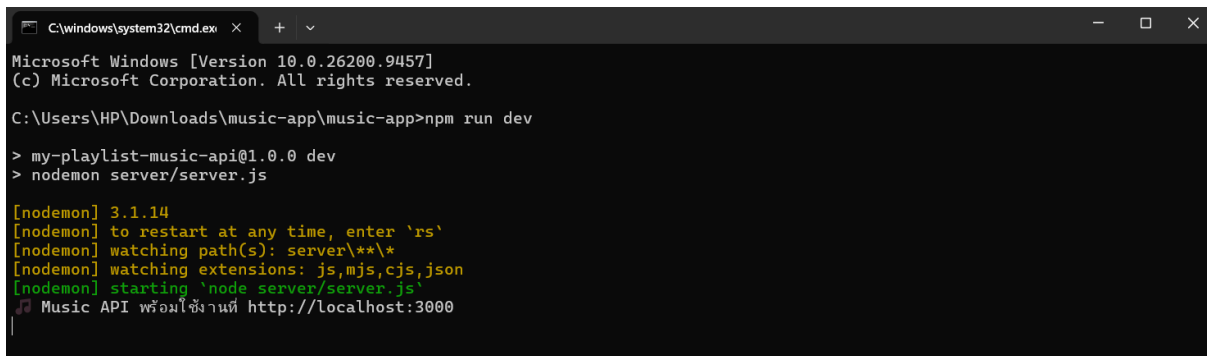
ส่วน	เครื่องมือ	หน้าที่
Runtime	Node.js (v18 ขึ้นไป)	รันโค้ด JavaScript ฟังเซิร์ฟเวอร์
Web framework	Express.js 5	สร้าง route ของ REST API และเสิร์ฟไฟล์หน้าเว็บ
เครื่องมือพัฒนา	nodemon 3	รีสตาร์ทเซิร์ฟเวอร์อัตโนมัติเมื่อแก้โค้ด (npm run dev)
ที่เก็บข้อมูล	ไฟล์ JSON	เก็บรายชื่อเพลงใน server/data/songs.json
Frontend	HTML, CSS, JavaScript	หน้าเว็บ ไม่ใช้ framework
ฟอนต์	Inter, Noto Sans Thai, Gochi Hand	เก็บไฟล์ไว้ใน public/fonts ไม่โหลดจากภายนอก
ทดสอบ API	curl ใน Command Prompt	ส่ง request ทดสอบทุก endpoint

4. โครงสร้างโปรเจกต์

- server/server.js สร้าง Express app โดยใช้ `express.json()` อ่าน body แบบ JSON และ `express.static()` เสิร์ฟไฟล์ในโฟลเดอร์ public ทำให้เปิดหน้าเว็บและเรียก API ได้จาก `http://localhost:3000` ที่เดียว
- server/data/songs.json เซิร์ฟเวอร์อ่านไฟล์นี้ตอนเริ่มทำงาน และเขียนทับทุกครั้งที่มีการเพิ่มแก้ไข หรือลบ ข้อมูลจึงไม่หายเมื่อปิดเซิร์ฟเวอร์
- public/app.js มีฟังก์ชันหลักคือ `loadSongs()`, `loadGenres()`, `renderSongs()` และตัวจัดการปุ่มต่าง ๆ
- package.json กำหนดคำสั่ง `npm start (node)` และ `npm run dev (nodemon)`

5. วิธีติดตั้งและรัน (Windows)

1. ติดตั้ง Node.js เวอร์ชัน LTS จาก <https://nodejs.org>
 2. เปิดโฟลเดอร์โปรเจกต์ (โฟลเดอร์ที่มีไฟล์ package.json) คลิกที่แถบ path ของ File Explorer พิมพ์ `cmd` แล้วกด Enter
 3. ติดตั้ง dependencies ด้วยคำสั่ง `npm install`
 4. รันเซิร์ฟเวอร์ด้วยคำสั่ง `npm run dev` เมื่อขึ้นข้อความ Music API พร้อมใช้งานที่ `http://localhost:3000` แปลว่าพร้อมใช้งาน
 5. เปิดเบราว์เซอร์ไปที่ <http://localhost:3000> และเปิดหน้าต่าง cmd ค้างไว้ระหว่างใช้งาน
- หมายเหตุ: ต้องเปิดผ่าน `http://localhost:3000` เท่านั้น ถ้าดับเบิลคลิกเปิดไฟล์ `index.html` ตรง ๆ หน้าเว็บจะเรียก API ไม่ได้ และจะแสดงข้อความแนะนำให้รันเซิร์ฟเวอร์ก่อน



```
C:\windows\system32\cmd.exe X + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\music-app\music-app>npm run dev

> my-playlist-music-api@1.0.0 dev
> nodemon server/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): server\**\*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server/server.js`
🔊 Music API พร้อมใช้งานที่ http://localhost:3000
```

ภาพที่ 1 รันเซิร์ฟเวอร์ด้วยคำสั่ง npm run dev ใน Command Prompt

nodemon เริ่มเซิร์ฟเวอร์ server/server.js และแสดงข้อความ Music API พร้อมใช้งานที่ <http://localhost:3000>
แปลว่าเปิดเว็บและเรียก API ได้แล้ว

6. การออกแบบข้อมูล

Resource หลักคือ **songs** แต่ละเพลงเก็บเป็น object ใน array ของไฟล์ songs.json มีโครงสร้างดังนี้

ฟิลด์	ชนิดข้อมูล	บังคับ	กฎการตรวจสอบ	ตัวอย่าง
id	number	สร้างอัตโนมัติ	ค่า id มากที่สุดที่มีอยู่ + 1 ผู้ใช้กำหนดเองไม่ได้	3
title	string	✓	ต้องเป็นข้อความ และไม่เป็นค่าว่างหลังตัดช่องว่างหัวท้าย	Friends
artist	string	✓	ต้องเป็นข้อความ และไม่เป็นค่าว่าง	Chase Atlantic
genre	string	✓	ต้องเป็นข้อความ และไม่เป็นค่าว่าง	Alternative R&B
spotifyUrl	string / null	–	ถ้าใส่ ต้องขึ้นต้นด้วย https://open.spotify.com/	null

ตัวอย่างข้อมูลเพลง 1 รายการ

```
{ "id": 3, "title": "Friends", "artist": "Chase Atlantic", "genre": "Alternative R&B", "spotifyUrl": null }
```

การอ่านและบันทึกข้อมูล

- ตอนเริ่มเซิร์ฟเวอร์ ฟังก์ชัน `loadSongs()` อ่านไฟล์ songs.json มาเก็บในตัวแปร songs (ถ้าอ่านไม่ได้จะเริ่มจาก array ว่าง)
- หลัง POST, PATCH หรือ DELETE สำเร็จ ฟังก์ชัน `saveSongs()` เขียนข้อมูลทั้งหมดกลับลงไฟล์
- id ของเพลงใหม่คำนวณจาก `Math.max(...ids) + 1` ทำให้ id ไม่ซ้ำกัน

7. การออกแบบ REST API

7.1 หลักการออกแบบ

- ใช้ path `/api/songs` เป็นค่านามพหุพจน์แทนกลุ่มของเพลง และ `/api/songs/:id` แทนเพลง 1 รายการ
- ใช้ HTTP method บอกการกระทำ: GET = อ่าน, POST = สร้าง, PATCH = แก้ไขบางส่วน, DELETE = ลบ
- ส่งและรับข้อมูลเป็น JSON ทั้งหมด (Content-Type: application/json)
- กรณีผิดพลาด ตอบกลับเป็น JSON ที่มีฟิลด์ `error` (และ `details` เมื่อมีหลายข้อผิดพลาด) เพื่อให้หน้าเว็บนำไปแสดงได้
- เลือกใช้ PATCH แทน PUT เพราะแก้ไขเฉพาะฟิลด์ที่ต้องการได้ โดยไม่ต้องส่งข้อมูลครบทุกฟิลด์

7.2 Status code ที่ใช้

Status code	ความหมาย	ใช้เมื่อ
200 OK	สำเร็จ	GET และ PATCH สำเร็จ
201 Created	สร้างข้อมูลใหม่สำเร็จ	POST สำเร็จ
204 No Content	สำเร็จ ไม่มีข้อมูลตอบกลับ	DELETE สำเร็จ
400 Bad Request	ข้อมูลที่ส่งมาไม่ถูกต้อง	ข้อมูลไม่ครบ ผิดรูปแบบ หรือ JSON ผิด
404 Not Found	ไม่พบข้อมูล	ไม่พบ id ที่ระบุ หรือเรียก endpoint ที่ไม่มี
500 Internal Server Error	ข้อผิดพลาดในเซิร์ฟเวอร์	เกิดข้อผิดพลาดที่ไม่คาดคิด

7.3 สรุป Endpoint

Method	Endpoint	หน้าที่	Status code
GET	/api/songs	ดึงเพลงทั้งหมด (กรองด้วย query string ได้)	200
GET	/api/songs/:id	ดึงเพลง 1 รายการ	200, 404
POST	/api/songs	เพิ่มเพลงใหม่	201, 400
PATCH	/api/songs/:id	แก้ไขบางฟิลด์ของเพลง	200, 400, 404
DELETE	/api/songs/:id	ลบเพลง	204, 404
GET	/api/genres	(เสริม) รายชื่อแนวเพลงพร้อมจำนวนเพลง	200

7.4 รายละเอียดแต่ละ Endpoint

1) GET /api/songs ดึงรายการเพลงทั้งหมด

Query string	ตัวอย่าง	ผลลัพธ์
genre	?genre=Pop	เฉพาะแนวเพลงนั้น (ไม่สนตัวพิมพ์เล็ก/ใหญ่)
artist	?artist=taylor	เฉพาะศิลปินที่มีคำนี้ในชื่อ
q	?q=love	ค้นหาคำนี้ในชื่อเพลงหรือชื่อศิลปิน
sort	?sort=title / ?sort=artist	เรียงตามชื่อเพลง หรือชื่อศิลปิน (A-Z)

- ใช้หลายพารามิเตอร์ร่วมกันได้ เช่น /api/songs?genre=Pop&q=taylor&sort=title
- ขั้นตอนในเซิร์ฟเวอร์: คัดลอก array เพลง → กรองที่ละเอียดขึ้นด้วย filter() → เรียงด้วย sort() → ตอบกลับด้วย res.json()
- การเรียงตามตัวอักษรใช้ localeCompare(..., { sensitivity: 'base', numeric: true }) จึงไม่สนตัวพิมพ์เล็ก/ใหญ่ (เช่น exile อยู่ระหว่าง E) และเรียงตัวเลขก่อนตัวอักษร (1989 อยู่ลำดับแรก)
- ถ้าไม่มีเพลงตรงเงื่อนไข จะได้ array ว่าง [] พร้อมสถานะ 200

```
GET /api/songs?genre=Indie%20Rock
→ 200 OK
[ { "id": 14, "title": "Bad Dream", "artist": "Wallows", "genre": "Indie Rock", "spotify
Url": null },
  { "id": 26, "title": "End of Beginning", "artist": "Djo", ... },
  { "id": 31, "title": "I Wanna Be Yours", "artist": "Arctic Monkeys", ... } ]
```

2) GET /api/songs/:id ดึงเพลง 1 รายการด้วย route parameter

- แปลง req.params.id เป็นตัวเลข แล้วค้นหาด้วย find()
- ถ้าพบ ตอบ 200 พร้อมข้อมูลเพลง ถ้าไม่พบ ตอบ 404

```
GET /api/songs/3 → 200 OK { "id": 3, "title": "Friends", "artist": "Chase Atl
antic", ... }
GET /api/songs/999 → 404 Not Found { "error": "ไม่พบเพลง id 999" }
```

3) POST /api/songs เพิ่มเพลงใหม่

- ตรวจสอบข้อมูลด้วย validateSong() ถ้ามีข้อผิดพลาดจะรวบรวมทุกข้อแล้วตอบ 400 ครั้งเดียว
- ถ้าผ่าน ใช้ pickFields() เลือกเฉพาะฟิลด์ที่อนุญาตและตัดช่องว่างหัวท้าย สร้าง id ใหม่ แล้วบันทึกลงไฟล์
- ตอบ 201 Created พร้อมข้อมูลเพลงที่สร้าง

```
POST /api/songs
Content-Type: application/json
{ "title": "Sweater Weather", "artist": "The Neighbourhood", "genre": "Indie Rock" }
→ 201 Created
{ "id": 34, "spotifyUrl": null, "title": "Sweater Weather", "artist": "The Neighbourhood
", "genre": "Indie Rock" }

POST /api/songs
{ "title": "Sweater Weather" }
→ 400 Bad Request
{ "error": "ข้อมูลไม่ครบหรือไม่ถูก
ต้อง", "details": ["artist (ศิลปิน) ห้ามว่าง", "genre (แนวเพลง) ห้ามว่าง"] }
```

4) PATCH /api/songs/:id แก้ไขเฉพาะฟิลด์ที่ส่งมา

- หาเพลงจาก id ก่อน ถ้าไม่พบตอบ 404
- ตรวจสอบแบบ partial คือตรวจเฉพาะฟิลด์ที่ส่งมา ฟิลด์ที่ไม่ได้ส่งจะคงค่าเดิม
- รวมข้อมูลเดิมกับข้อมูลใหม่ด้วย { ...เดิม, ...ใหม่ } แล้วบันทึก ตอบ 200 พร้อมข้อมูลล่าสุด

```
PATCH /api/songs/34
{ "genre": "Alternative Rock" }
→ 200 OK { "id": 34, "title": "Sweater Weather", ..., "genre": "Alternative Rock" }

PATCH /api/songs/999
→ 404 Not Found { "error": "ไม่พบเพลง id 999" }
```

5) DELETE /api/songs/:id ลบเพลง

- หาเพลงจาก id ถ้าไม่พบตอบ 404
- ถ้าพบ ลบออกจาก array ด้วย splice() บันทึกไฟล์ แล้วตอบ 204 No Content (ไม่มี body)

```
DELETE /api/songs/34 → 204 No Content
DELETE /api/songs/34 → 404 Not Found { "error": "ไม่พบเพลง id 34" } (ลบซ้ำ)
```

6) GET /api/genres (endpoint เสริม) นับจำนวนเพลงในแต่ละแนวเพลง ใช้สร้างโฟลเดอร์แนวเพลงในหน้าเว็บ

```
GET /api/genres → 200 OK [ { "name": "Indie Pop", "count": 5 }, { "name": "Alternative R&B", "count": 8 }, ... ]
```

7.5 การตรวจสอบข้อมูลและจัดการข้อผิดพลาด

- ฟังก์ชัน validateSong(body, partial) ใช้ร่วมกันทั้ง POST (ตรวจทุกฟิลด์ที่บังคับ) และ PATCH (ตรวจเฉพาะฟิลด์ที่ส่งมา)
- ส่ง JSON ผิดรูปแบบ เช่น ลืมปีกกา เซิร์ฟเวอร์ตอบ 400 { "error": "JSON ไม่ถูกต้อง" }
- เรียก path ที่ขึ้นต้นด้วย /api แต่ไม่มีอยู่จริง ตอบ 404 { "error": "ไม่พบ endpoint นี้" }
- ข้อผิดพลาดอื่นที่ไม่คาดคิด ตอบ 500 และแสดงรายละเอียดใน console ของเซิร์ฟเวอร์
- เซิร์ฟเวอร์รับเฉพาะฟิลด์ที่กำหนด (title, artist, genre, spotifyUrl) ฟิลด์อื่นที่ส่งมาจะถูกละทิ้ง

8. ฝั่งไคลเอนต์ (Frontend)

8.1 ส่วนประกอบของหน้าเว็บ

- ปก playlist ภาพปกวาดด้วย SVG พร้อมชื่อ My Playlist และปุ่ม Play / Shuffle
- ช่องค้นหา ตัวเลือกการเรียง และปุ่ม + เพิ่มเพลง อยู่ใต้ปก
- การ์ดแจ้งเตือน สรุปจำนวนเพลงและแนวเพลง
- โพลเดอร์แนวเพลง ไอคอนโพลเดอร์พร้อมตัวเลขสีแดงบอกจำนวนเพลง กดเพื่อกรอง
- รายการเพลง การ์ดแต่ละเพลงมีลำดับ ปก ชื่อเพลง ศิลปิน แนวเพลง และปุ่ม ▶ Spotify / 🗑️ แก้ไข / 🗑️ ลบ
- แถบ Now playing ด้านล่าง แสดงเพลงที่เลือก ปุ่มก่อนหน้านี้ / เล่น / ถัดไป และปุ่มเปิด Spotify
- ฟอรัม popup สำหรับเพิ่มและแก้ไขเพลง และการแจ้งเตือนที่เลื่อนลงมาเมื่อทำการสำเร็จ

8.2 การเรียก API ด้วย fetch()

ดึงรายการเพลง สร้าง query string จากตัวกรองที่เลือกอยู่ แล้วเรียก GET

```
async function loadSongs() {
  const params = new URLSearchParams();
  if (activeGenre) params.set('genre', activeGenre);
  if ($('#search').value.trim()) params.set('q', $('#search').value.trim());
  if ($('#sort').value) params.set('sort', $('#sort').value);

  const res = await fetch(`${API}?${params}`);
  current = await res.json();
  renderSongs(current);
}
```

เพิ่ม / แก้ไขเพลง ใช้ฟอร์มเดียวกัน ถ้ามี id จะเรียก PATCH ถ้าไม่มีจะเรียก POST

```
const res = await fetch(id ? `${API}/${id}` : API, {
  method: id ? 'PATCH' : 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(body),
});
const data = await res.json();
if (!res.ok) { // 400 → ไซส์ไฟล์ของที่ยังไม่ได้กรอก
  markInvalid(data.details || []);
  return;
}
refresh(); // 201 / 200 → โหลดรายการใหม่ทันที
```

ลบเพลง ถามยืนยันก่อน แล้วเรียก DELETE เมื่อได้ 204 จึงโหลดรายการใหม่

```
if (!confirm('ลบเพลง "${title}" ใช่ไหม?')) return;
const res = await fetch(`${API}/${id}`, { method: 'DELETE' });
if (res.status === 204) { toast('ลบเพลง "${title}" ออกจาก playlist แล้ว'); refresh(); }
```

8.3 ลำดับการทำงานของแต่ละจุด

- เปิดหน้าเว็บ: เรียก GET /api/genres เพื่อสร้างโพลเดอร์ → เรียก GET /api/songs เพื่อแสดงรายการ
- กดโพลเดอร์แนวเพลง: เก็บแนวเพลงที่เลือก → เรียก GET /api/songs?genre=... → แสดงเฉพาะเพลงแนวนั้น
- พิมพ์ค้นหา: รอ 250 มิลลิวินาทีหลังหยุดพิมพ์ (ลดจำนวน request) → เรียก GET /api/songs?q=...
- เพิ่มเพลง: กด + เพิ่มเพลง → กรอกฟอร์ม → POST → ถ้า 400 ไซส์ไฟล์ของที่ยังไม่กรอกเป็นสีชมพู / ถ้า 201 ปิดฟอร์ม แจ้งเตือน และโหลดรายการใหม่
- แก้ไขเพลง: กด 🗑️ → GET /api/songs/:id มาใส่ฟอร์ม → แก้ไข → PATCH → แจ้งเตือนและโหลดรายการใหม่
- ลบเพลง: กด 🗑️ → ยืนยัน → DELETE → ได้ 204 → แจ้งเตือนและโหลดรายการใหม่

8.4 การลิงก์ไป Spotify

- กดชื่อเพลง ปุ่ม ▶ สีเขียว หรือที่การ์ดเพลง จะเปิด Spotify ในแท็บใหม่
- ถ้าเพลงมี spotifyUrl จะเปิดลิงก์นั้นตรง ๆ ถ้าไม่มีจะเปิดหน้าค้นหา `open.spotify.com/search/ชื่อเพลง คิลปิน`
- ปุ่ม Play เปิดเพลงแรกในรายการ ปุ่ม Shuffle สุ่มเพลงจากรายการที่กรองอยู่
- เป็นเพียงลิงก์ให้ผู้ซัดไปฟังเอง หน้าเว็บไม่ได้ดึงข้อมูลจาก API ของ Spotify จึงยังเป็นไปตามข้อกำหนดที่ให้เรียก API ของตัวเองเท่านั้น

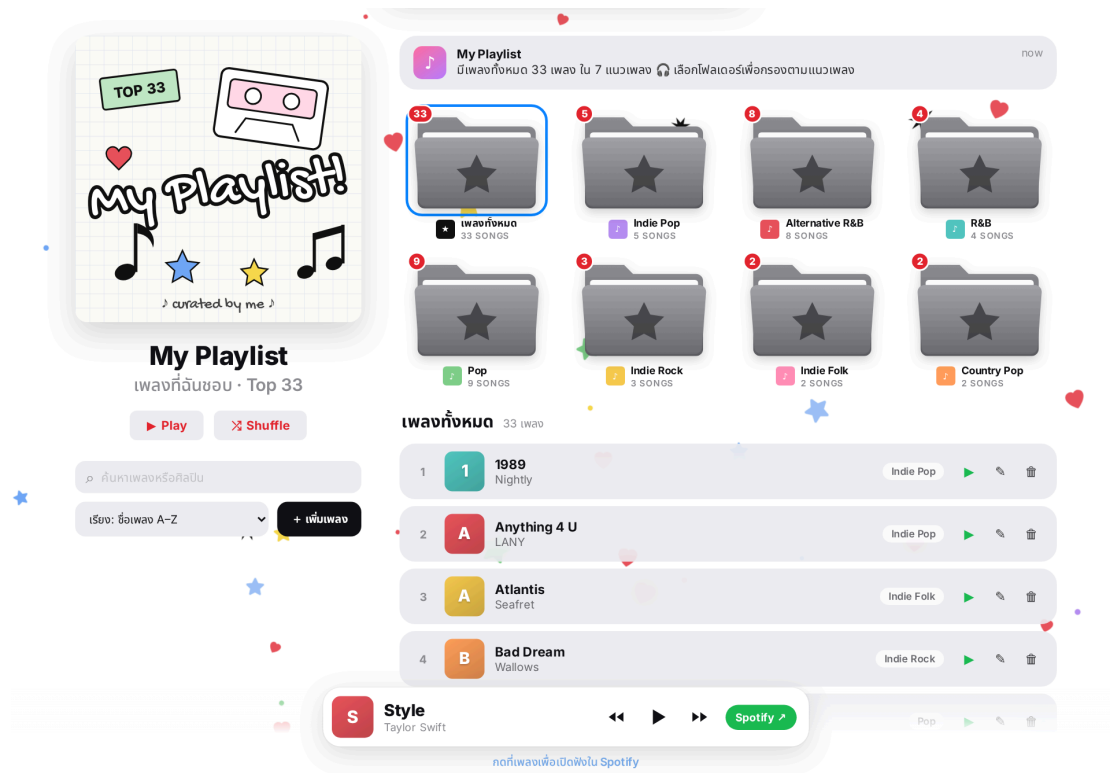
8.5 การออกแบบหน้าตา

- คีบอร์ดแสดงด้วยลายดอตเดิลดาว หัวใจ และจุดสีพาสเทล (ฟ้า เขียว เหลือง แดง) สร้างด้วย JavaScript แบบกำหนด seed ทำให้ลายเหมือนเดิมทุกครั้ง
- การ์ดและแถบต่าง ๆ ออกแบบให้คล้ายหน้าจอล็อกของ iOS คือมูมโค้งมน พื้นเทาโปร่งแสง แบบเบลอ (backdrop-filter)
- จัดวางแบบ 2 คอลัมน์ด้วย CSS Grid ข่ายเป็นปก playlist ขวาคือโฟลเดอร์และรายการเพลง
- สีปกของแต่ละเพลงเลือกจากชุดสีพาสเทลตามชื่อเพลง ทำให้เพลงเดิมได้สีเดิมเสมอ

8.6 ความปลอดภัยและการใช้งาน

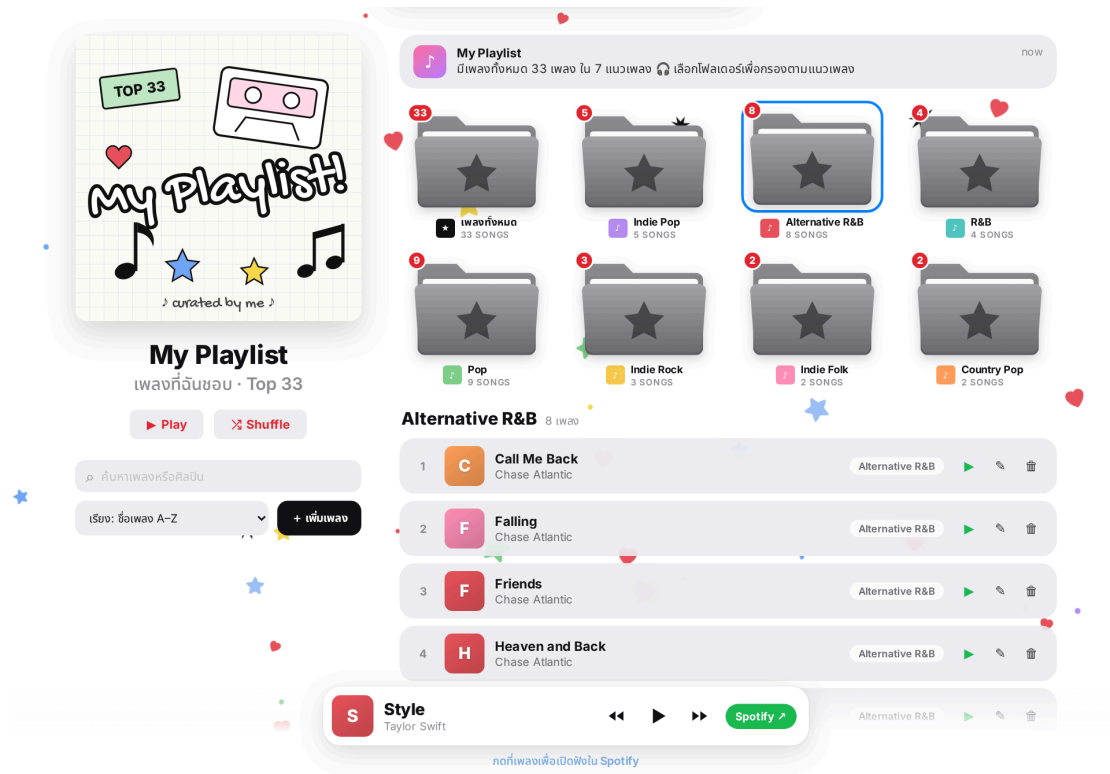
- `escape` ข้อความทุกครั้งก่อนแทรกลง HTML ด้วยฟังก์ชัน `escapeHtml()` เพื่อป้องกัน XSS
- ตรวจสอบข้อมูลที่ฝั่งเซิร์ฟเวอร์เสมอ แม้หน้าเว็บจะมีฟอร์มก็ตาม
- เมื่อเซิร์ฟเวอร์ตอบ 400 หน้าเว็บอ่านรายการ details เพื่อหาว่าฟิลด์ไหนผิด แล้วไฮไลต์ช่องนั้นเป็นสีแดงและเลื่อนเคอร์เซอร์ไปช่องแรกที่ผิด แทนการแสดงความยาว ๆ และไฮไลต์จะหายเมื่อเริ่มพิมพ์
- ถ้าเชื่อมต่อเซิร์ฟเวอร์ไม่ได้ หน้าเว็บจะแสดงความบกพร่องวิธีรันเซิร์ฟเวอร์แทนการแสดงผลหน้าว่าง

9. ภาพการทำงานของเว็บไซต์



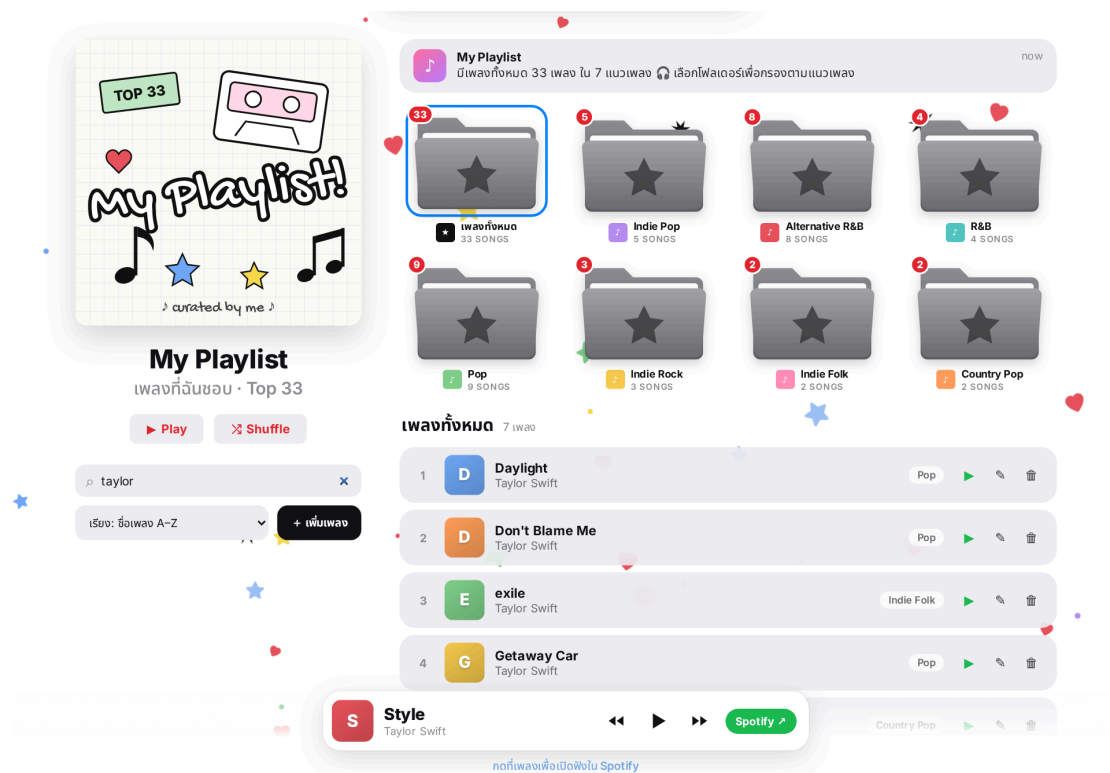
ภาพที่ 2 หน้าแรกของเว็บไซต์

หน้าเว็บเรียก GET /api/genres เพื่อสร้างโฟลเดอร์ 8 โฟลเดอร์ (ทั้งหมด + 7 แนวเพลง) พร้อมตัวเลขจำนวนเพลง และเรียก GET /api/songs?sort=title เพื่อแสดงรายการเพลงทั้ง 33 เพลงเรียงตามตัวอักษร แถบ Now playing ด้านล่างแสดงเพลงที่เลือกล่าสุด



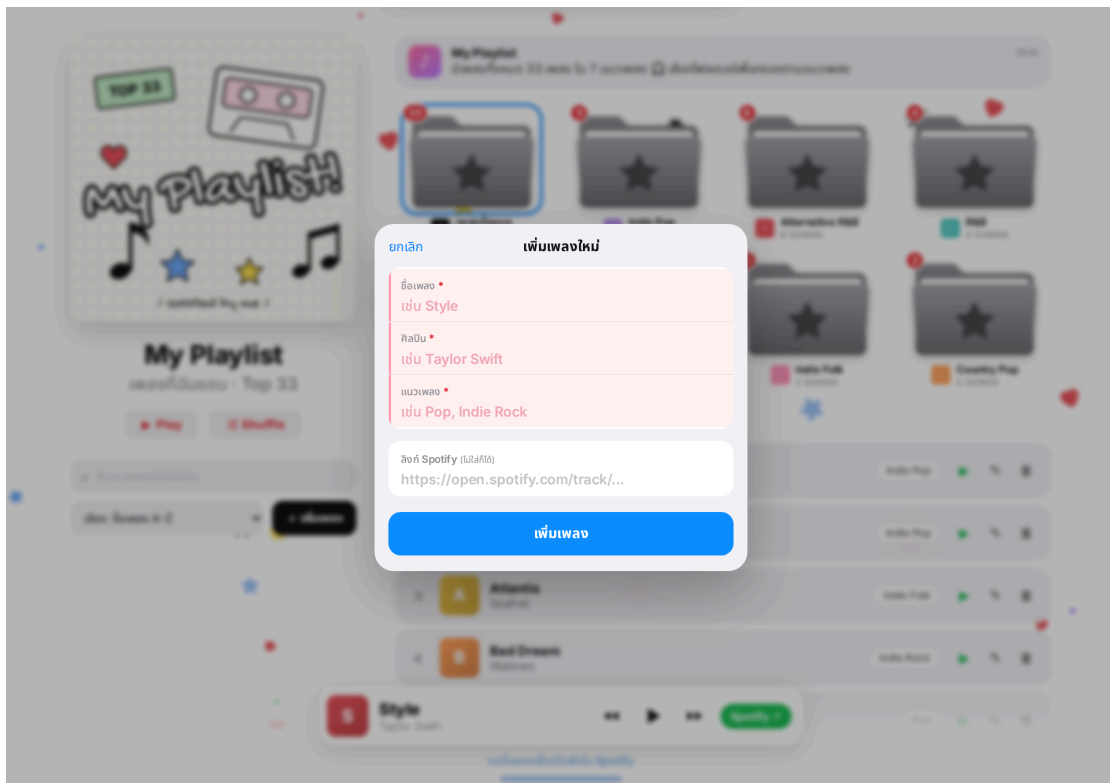
ภาพที่ 3 กรองเพลงตามแนวเพลง Alternative R&B

เมื่อกดโฟลเดอร์ Alternative R&B หน้าเว็บเรียก GET /api/songs?genre=Alternative%20R%26B และแสดงเฉพาะ 8 เพลงในแนวนั้น โดยไม่โหลดหน้าใหม่



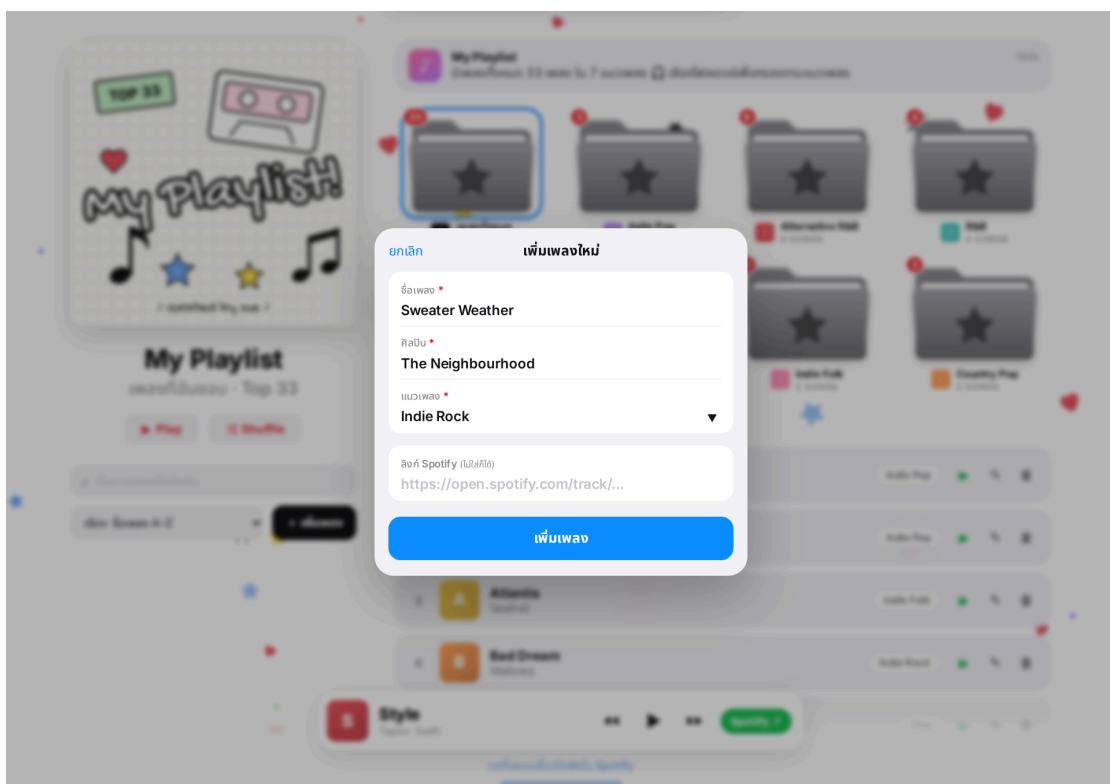
ภาพที่ 4 ค้นหาเพลงด้วยคำว่า “taylor”

เมื่อพิมพ์ในช่องค้นหา หน้าเว็บเรียก GET /api/songs?q=taylor และแสดงเพลงของ Taylor Swift ทั้ง 7 เพลง



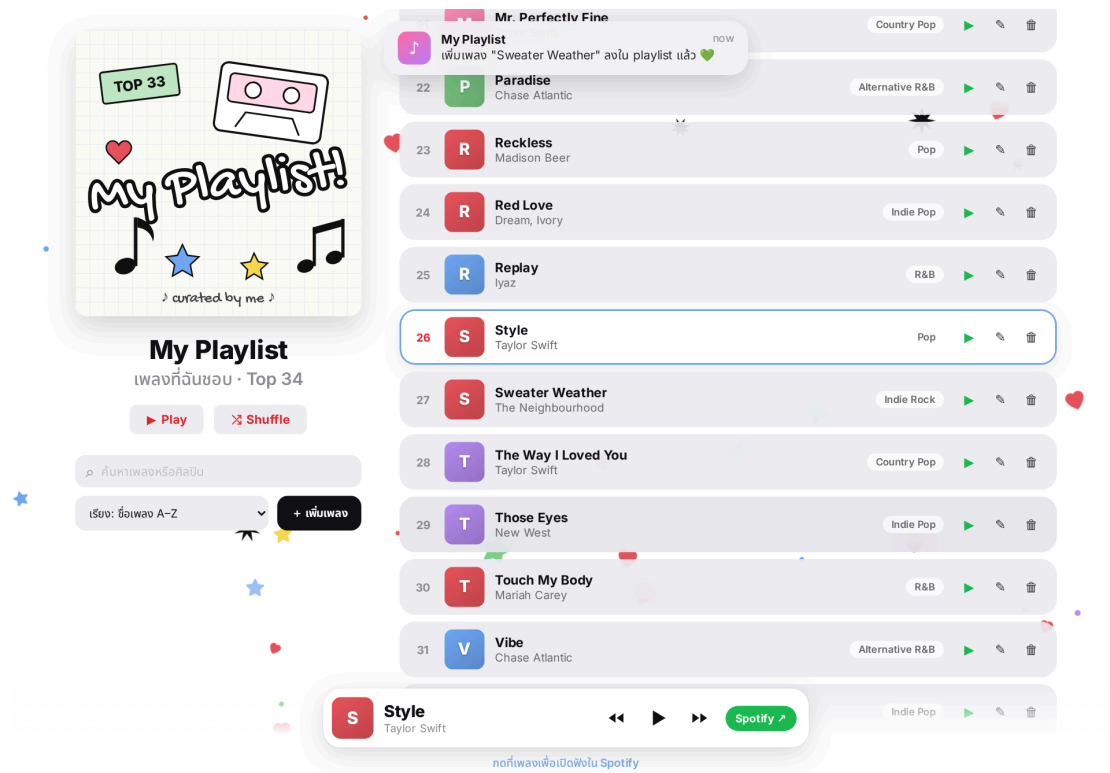
ภาพที่ 5 กดเพิ่มเพลงโดยไม่กรอกข้อมูล

ฟอร์มส่ง POST ที่ไม่มีข้อมูล เซิร์ฟเวอร์ตอบ 400 Bad Request พร้อมรายการข้อผิดพลาด หน้าเว็บจึงไฮไลต์ช่องชื่อเพลง ศิลปิน และแนวเพลงที่ยังว่างเป็นสีชมพู



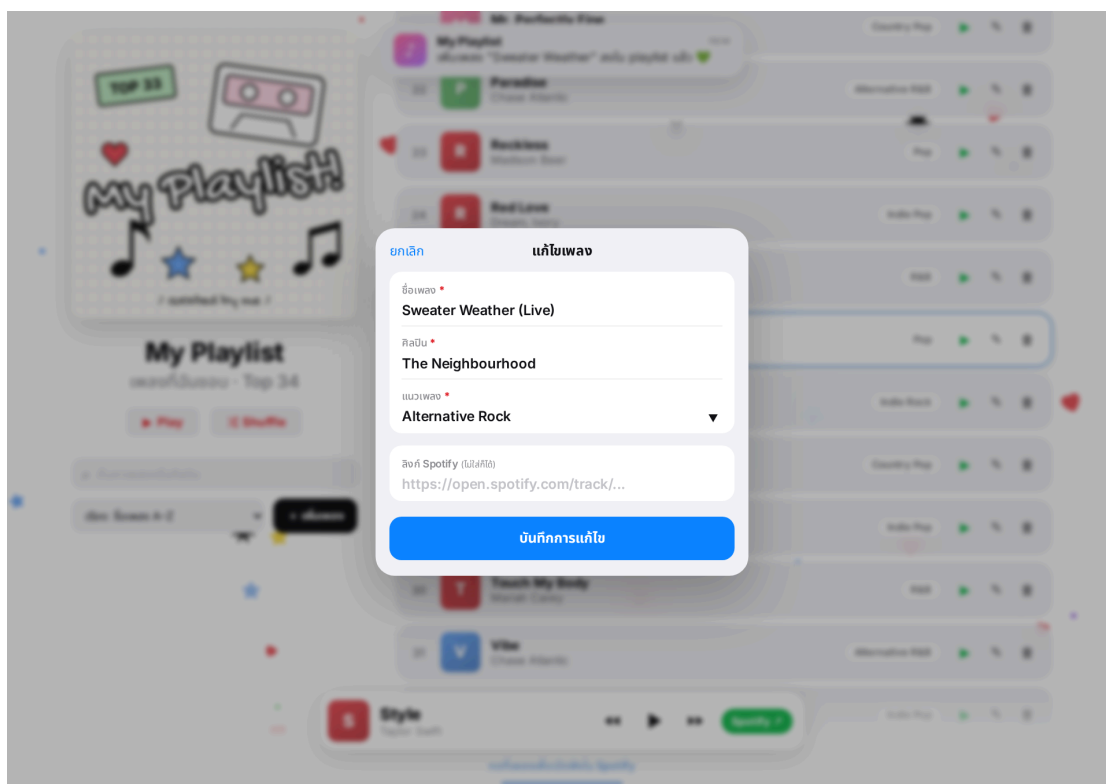
ภาพที่ 6 กรอกข้อมูลเพลงใหม่ในฟอร์ม

กรอกชื่อเพลง Sweater Weather ศิลปิน The Neighbourhood แนวเพลง Indie Rock ช่องแนวเพลงมีรายการแนะนำจากแนวเพลงที่มีอยู่



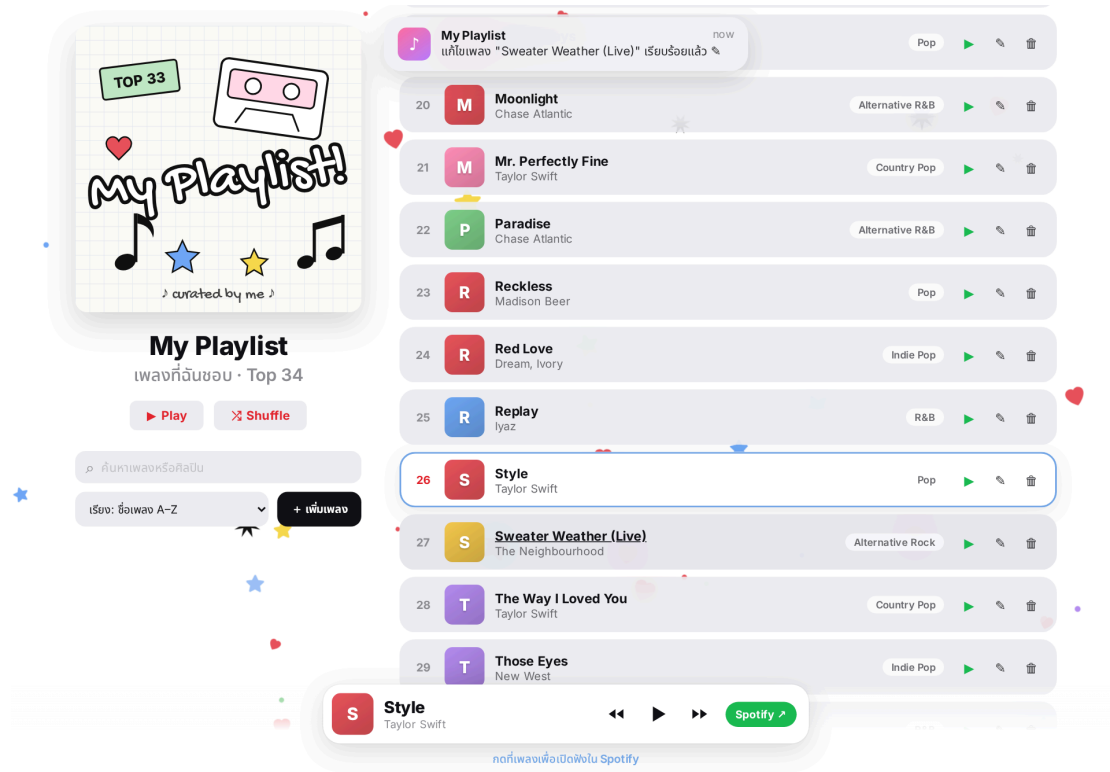
ภาพที่ 7 หลังเพิ่มเพลงสำเร็จ

เซิร์ฟเวอร์ตอบ 201 Created เพลงใหม่ปรากฏในรายการทันทีตามลำดับตัวอักษร (หมวด S) และมีการแจ้งเตือนเลื่อนลงมาจากด้านบน



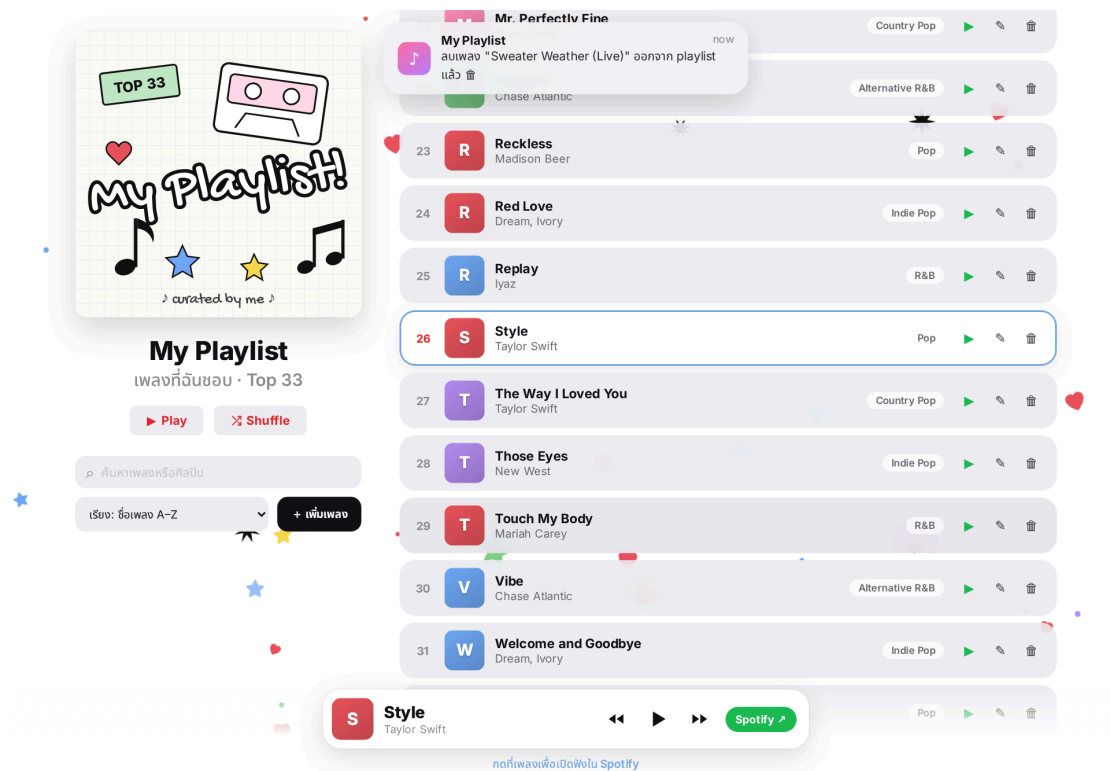
ภาพที่ 8 แก้ไขเพลง

กดปุ่ม หน้าเว็บเรียก GET /api/songs/34 มาใส่ฟอร์ม แล้วแก้ไขเพลงเป็น Sweater Weather (Live) และแนวเพลงเป็น Alternative Rock



ภาพที่ 9 หลังแก้ไขเพลงสำเร็จ

หน้าเว็บส่ง PATCH /api/songs/34 ได้ 200 OK ข้อมูลในรายการเปลี่ยนตามที่แก้ไข



ภาพที่ 10 หลังลบเพลง

กดปุ่ม 🗑️ และยืนยัน หน้าเว็บส่ง DELETE /api/songs/34 ได้ 204 No Content เพลงหายไปจากรายการและมีการแจ้งเตือน

10. การทดสอบ API

ทดสอบทุก endpoint ด้วยคำสั่ง curl ใน Command Prompt ของ Windows ทั้งกรณีสำเร็จและกรณีผิดพลาด ผลการทดสอบเป็นดังนี้

#	กรณีทดสอบ	Request	ผลที่คาดเดา	ผลจริง
1	กรองตามแนวเพลง	GET /api/songs?genre=Indie%20Rock	200 และได้ 3 เพลง	ผ่าน
2	ค้นหาและเรียงลำดับ	GET /api/songs?q=taylor&sort=title	200 และได้ 7 เพลงเรียงตามชื่อ	ผ่าน
3	ดึงเพลงที่มีอยู่	GET /api/songs/3	200 ได้เพลง Friends	ผ่าน
4	ดึงเพลงที่ไม่มี	GET /api/songs/999	404 Not Found	ผ่าน
5	เพิ่มเพลงข้อมูลไม่ครบ	POST /api/songs (มีแค่ title)	400 พร้อมรายการ error	ผ่าน
6	เพิ่มเพลงข้อมูลครบ	POST /api/songs	201 Created ได้ id 34	ผ่าน
7	แก้ไขเพลงที่มีอยู่	PATCH /api/songs/34	200 ข้อมูลเปลี่ยน	ผ่าน
8	แก้ไขเพลงที่ไม่มี	PATCH /api/songs/999	404 Not Found	ผ่าน
9	ลบเพลง	DELETE /api/songs/34	204 No Content	ผ่าน
10	ลบเพลงซ้ำ	DELETE /api/songs/34	404 Not Found	ผ่าน
11	ส่งลิงก์ Spotify ผิดรูปแบบ	POST spotifyUrl = https://evil.com/x	400 พร้อม error	ผ่าน

```
C:\windows\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\music-app\music-app>curl -i -s "http://localhost:3000/api/songs?genre=Indie%20Rock"
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[{"id":14,"title":"Bad Dream","artist":"Wallows","genre":"Indie Rock","spotifyUrl":null}, {"id":26,"title":"End of Beginning","artist":"Djo","genre":"Indie Rock","spotifyUrl":null}, {"id":31,"title":"I Wanna Be Yours","artist":"Arctic Monkeys","genre":"Indie Rock","spotifyUrl":null}]

C:\Users\HP\Downloads\music-app\music-app>curl -s "http://localhost:3000/api/songs?q=taylor&sort=title"
[{"id":19,"title":"Daylight","artist":"Taylor Swift","genre":"Pop","spotifyUrl":null}, {"id":21,"title":"Don't Blame Me","artist":"Taylor Swift","genre":"Pop","spotifyUrl":null}, {"id":17,"title":"Exile","artist":"Taylor Swift","genre":"Indie Folk","spotifyUrl":null}, {"id":22,"title":"Getaway Car","artist":"Taylor Swift","genre":"Pop","spotifyUrl":null}, {"id":20,"title":"Mr. Perfectly Fine","artist":"Taylor Swift","genre":"Country Pop","spotifyUrl":null}, {"id":16,"title":"Style","artist":"Taylor Swift","genre":"Pop","spotifyUrl":null}, {"id":18,"title":"The Way I Loved You","artist":"Taylor Swift","genre":"Country Pop","spotifyUrl":null}]

C:\Users\HP\Downloads\music-app\music-app>
```

ภาพที่ 11 ทดสอบ GET /api/songs พร้อม query string

คำสั่งแรกกรองแนวเพลง Indie Rock ได้ 3 เพลง คำสั่งที่สองค้นหา taylor และเรียงตามชื่อเพลง

```
C:\Windows\system32\cmd.exe x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\music-app\music-app>curl -i -s http://localhost:3000/api/songs/3
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"id":3,"title":"Friends","artist":"Chase Atlantic","genre":"Alternative R&B","spotifyUrl":null}

C:\Users\HP\Downloads\music-app\music-app>curl -i -s http://localhost:3000/api/songs/999
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{"error":"ไม่พบเพลง id 999"}

C:\Users\HP\Downloads\music-app\music-app>|
```

ภาพที่ 12 ทดสอบ GET /api/songs/:id
id 3 ได้ 200 OK พร้อมข้อมูลเพลง ส่วน id 999 ได้ 404 Not Found

```
C:\Windows\system32\cmd.exe x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\music-app\music-app>curl -i -s -X POST http://localhost:3000/api/songs -H "Content-Type: application/json" -d '{"title":"Sweater Weather"}'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"error":"ข้อมูลไม่ครบหรือไม่ถูกต้อง","details":["artist (ศิลปิน) ห้ามว่าง","genre (แนวเพลง) ห้ามว่าง"]}

C:\Users\HP\Downloads\music-app\music-app>curl -i -s -X POST http://localhost:3000/api/songs -H "Content-Type: application/json" -d '{"title":"Sweater Weather","artist":"The Neighbourhood","genre":"Indie Rock"}'
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"id":34,"spotifyUrl":null,"title":"Sweater Weather","artist":"The Neighbourhood","genre":"Indie Rock"}

C:\Users\HP\Downloads\music-app\music-app>|
```

ภาพที่ 13 ทดสอบ POST /api/songs
ส่งเฉพาะ title ได้ 400 Bad Request ส่งข้อมูลครบได้ 201 Created

```
C:\Windows\system32\cmd.exe x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\music-app\music-app>curl -i -s -X PATCH http://localhost:3000/api/songs/34 -H "Content-Type: application/json" -d '{"genre":"Alternative Rock"}'
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"id":34,"spotifyUrl":null,"title":"Sweater Weather","artist":"The Neighbourhood","genre":"Alternative Rock"}

C:\Users\HP\Downloads\music-app\music-app>curl -i -s -X PATCH http://localhost:3000/api/songs/999 -H "Content-Type: application/json" -d '{"genre":"Pop"}'
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{"error":"ไม่พบเพลง id 999"}

C:\Users\HP\Downloads\music-app\music-app>|
```

ภาพที่ 14 ทดสอบ PATCH /api/songs/:id
แก้แนวเพลงของ id 34 ได้ 200 OK ส่วน id 999 ได้ 404 Not Found

```
C:\Windows\system32\cmd.exe x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\music-app\music-app>curl -i -s -X DELETE http://localhost:3000/api/songs/34
HTTP/1.1 204 No Content

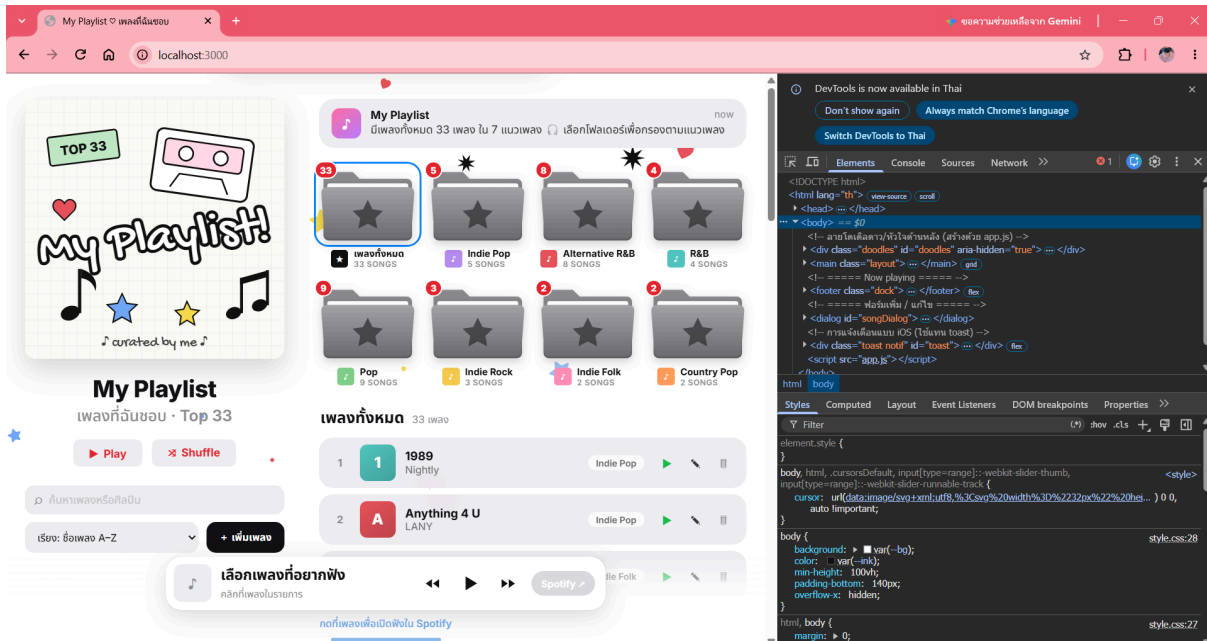
C:\Users\HP\Downloads\music-app\music-app>curl -i -s -X DELETE http://localhost:3000/api/songs/34
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
{"error": "ไม่พบเพลง id 34"}

C:\Users\HP\Downloads\music-app\music-app>
```

ภาพที่ 15 ทดสอบ DELETE /api/songs/:id
ลบ id 34 ได้ 204 No Content และลบซ้ำได้ 404 Not Found

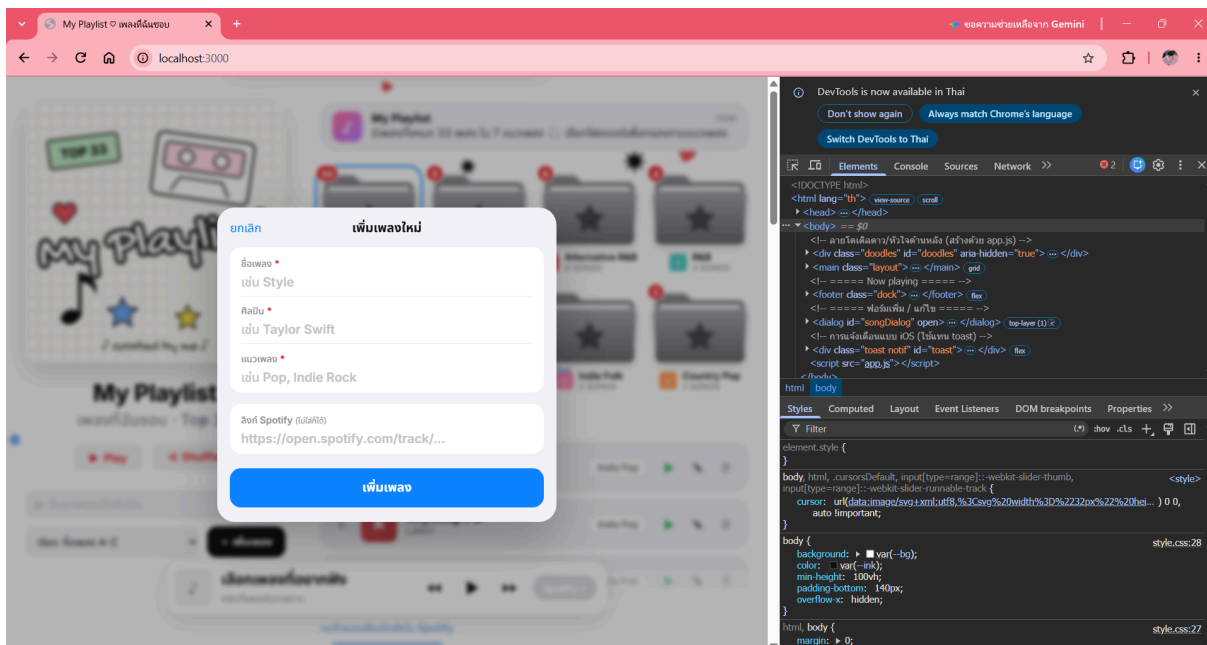
11. หลักฐานการดีบั๊ก

ใช้ Chrome DevTools (กด F12) ตรวจสอบการทำงานของหน้าเว็บขณะรันเซิร์ฟเวอร์ด้วยคำสั่ง `npm run dev` บนเครื่องจริง



ภาพที่ 16 ตรวจสอบโครงสร้างหน้าเว็บด้วย DevTools (Elements)

เปิดเว็บที่ `http://localhost:3000` แล้วกด F12 เปิด Chrome DevTools แท็บ Elements เพื่อตรวจสอบโครงสร้าง DOM ว่าส่วนประกอบหลักถูกสร้างครบ ได้แก่ ลายโดดเดิล (`div.doodles`), เนื้อหาหลัก (`main.layout`), แถบ Now playing (`footer.dock`), ฟอรัม (`dialog#songDialog`) และการแจ้งเตือน (`div#toast`) และหน้าเว็บดึงข้อมูลจาก API ได้ครบ 33 เพลง 7 แนวเพลง



ภาพที่ 17 ตรวจสอบการเปิดฟอรัมเพิ่มเพลง (dialog)

กดปุ่ม + เพิ่มเพลง แล้วตรวจสอบในแท็บ Elements ว่า `<dialog id="songDialog">` มี attribute `open` และอยู่ใน `top-layer` แสดงว่าฟอรัมเปิดด้วย `showModal()` ถูกต้อง หากหลังจากถูกเบลอ และกดส่วนอื่นของหน้าไม่ได้ระหว่งกรอกข้อมูล

12. รายชื่อเพลงในแอป

ข้อมูลเริ่มต้น 33 เพลง แบ่งเป็น 7 แนวเพลง ได้แก่ Indie Pop (5), Alternative R&B (8), R&B (4), Pop (9), Indie Rock (3), Indie Folk (2), Country Pop (2)

#	ชื่อเพลง	ศิลปิน	แนวเพลง
1	Anything 4 U	LANY	Indie Pop
2	1989	Nightly	Indie Pop
3	Friends	Chase Atlantic	Alternative R&B
4	Call Me Back	Chase Atlantic	Alternative R&B
5	Vibe	Chase Atlantic	Alternative R&B
6	Into It	Chase Atlantic	Alternative R&B
7	Paradise	Chase Atlantic	Alternative R&B
8	Moonlight	Chase Atlantic	Alternative R&B
9	Falling	Chase Atlantic	Alternative R&B
10	Heaven and Back	Chase Atlantic	Alternative R&B
11	Double Take	dhruv	R&B
12	Million Ways	HRVY	Pop
13	Break My Heart	Dua Lipa	Pop
14	Bad Dream	Wallows	Indie Rock
15	Domino	Jessie J	Pop
16	Style	Taylor Swift	Pop
17	exile	Taylor Swift	Indie Folk
18	The Way I Loved You	Taylor Swift	Country Pop
19	Daylight	Taylor Swift	Pop
20	Mr. Perfectly Fine	Taylor Swift	Country Pop
21	Dont Blame Me	Taylor Swift	Pop
22	Getaway Car	Taylor Swift	Pop
23	Red Love	Dream, Ivory	Indie Pop
24	Welcome and Goodbye	Dream, Ivory	Indie Pop
25	Reckless	Madison Beer	Pop
26	End of Beginning	Djo	Indie Rock
27	Replay	Iyaz	R&B
28	Will You Be My Valentine	Valen	Pop
29	Those Eyes	New West	Indie Pop
30	Atlantis	Seafret	Indie Folk
31	I Wanna Be Yours	Arctic Monkeys	Indie Rock
32	Touch My Body	Mariah Carey	R&B
33	You Da One	Rihanna	R&B